Sequences and Containers Meta

Computer Science Mentors 61A

September 30 – October 3, 2025

Note: Feel free to ask students about what topics they'd like to go through, since we have more questions in the worksheet than usual because many concepts were covered in the last week. **Example Timeline**

- Ice Breaker [5 min]

  If you want, start off with an icebreaker of your choice!

  Also, there is a link to a feedback form at the bottom of this worksheet! It would be amazing if you could highlight this/fill it out yourself so we can improve the worksheets in the future!
- Mini-lecture: Strings [3 min]
- Q1: Strings WWPD [5 min]
- Q2: Simple [5 min]
- Mini-lecture: list comprehensions [4 min]
- Q3: List Comprehensions [8 min]
- Mini-lecture: dictionaries [4 min]
- Q4: Dictionary Comprehension [7 min]
- Mini-lecture: Recursion With Lists [6 min]
- Q5: All True [7 min]
- Q6: Harder Recursion with Lists [if time]

Created by Anaisha Das and Tavisha Patel. Worksheet feedback: forms.gle/q67BZrHHfoKbF6V28

1 Strings Basics

1. What would Python display? Draw box-and-pointer diagrams for the following:

```
>>> x = 'CS61A '
>>> y = 'CS Mentors'
>>> x + y
```

```
'CS 61A CS Mentors'
```

```
>>> colors = ['red', 'yellow', 'green']
>>> 'green' in colors
```

```
True
```

```
>>> 'GREEN' in colors
```

```
False
```

```
>>> luggage = [c + ' suitcase' for c in colors]
>>> luggage
```

```
['red suitcase', 'yellow suitcase', 'green suitcase']
```

Teaching Tips

1. Make sure that they understand that strings are case sensitive for Python and how string concatenation works
2. The last question combines a lot of topics that they've recently learned so if short on time, feel free to skip to that.

Suggested Time: 5 min

2 List Basics

1. What would Python display? Draw box-and-pointer diagrams for the following:

```
>>> a = [1, 2, 3]
>>> a

[1, 2, 3]

>>> a[2]

        3

>>> a[:2]

[1, 2]

>>> b = [1, 2, 3, a, 4]
>>> b

[1, 2, 3, [1, 2, 3], 4]

>>> b[3][2]

3
```

Teaching Tips

1. Encourage students to draw box and pointer diagrams if they seem stuck.
2. Make sure to state that list adding two lists with + creates a new list. It may be helpful to draw a box and pointer diagram to show which statements make a new list.

Suggested Time: 3 min

2. Write a list comprehension that accomplishes each of the following tasks.

   (a) Square all the elements of a given list, `lst`.

   ```
   [x ** 2 for x in lst]
   ```

   (b) Compute the sum of all even numbers in the list `lst`. Is there a function you can apply to a list to get the sum of all elements?

   ```
   sum([x for x in lst if x % 2 == 0])
   ```

   (c) Return a list of lists such that `a = [[0], [0, 1], [0, 1, 2], [0, 1, 2, 3], [0, 1, 2, 3, 4]]`.

   ```
   a = [[x for x in range(y)] for y in range(1, 6)]
   ```

   (d) Return the same list as above, except now excluding every instance of the number 2: `b = [[0], [0, 1], [0, 1], [0, 1, 3], [0, 1, 3, 4]])`.

   ```
   b = [[x for x in range(y) if x != 2] for y in range(1, 6)]
   ```

Teaching Tips

1. It may be helpful to start with the basic list comprehension template of `[<expr> for <item> in <iterable> if <condition>]`
2. The list of list questions are tricky, so try nudging your students in the right direction by reminding them that it is completely possible to nest list comprehensions.

Suggested Time: 8 min

3 Dictionaries

1. Using a dictionary comprehension, complete the function below. It takes a single-argument function `f` and a dictionary `d`, and returns a new dictionary containing only the entries whose keys are even numbers, with `f` applied to their values.

```
def apply_to_even(f, d):
    '''
    >>> apply_to_even(lambda x: x**2, {1:3, 2:5, 4:2, 3:2})
    {2: 25, 4: 4}
    '''
    return {__ : _____ for ____ in ____ if ___________}
```

```
def apply_to_even(f, d):
    return {i: f(d[i]) for i in d if i % 2 == 0}
```

Suggested Time: 5 min

4 List Recursion

1. Complete the definition for `all_true`, which takes in a list `lst` and returns `True` if there are no Falsey values in the list and `False` otherwise. Make sure that your implementation is recursive. What is the name of the built-in Python function that does this?

```
def all_true(lst):
    '''
    >>> all_true([True, 1, 'True'])
    True
    >>> all_true([1, 0, 1])
    False
    >>> all_true([])
    True
    '''


    if not lst:
        return True
    elif not lst[0]:
        return False
    else:
        return all_true(lst[1:])

    The built-in Python function all does the same thing that all_true
       does!
```

Teaching Tips

1. It may be helpful to review recursion and discuss what the base case and recursive case should be.

Suggested Time: 7 min

2. Given a list of integers `lst`, return the maximum sum of a subset of size `n`. If `n` is greater than or equal to the length of `lst`, just return the sum of the elements `lst`.

```
def max_subset_sum(lst, n):
    '''
    >>> max_subset_sum([1, 2, 3, 4], 2)
    7
    >>> max_subset_sum([1, 4, 2, 0, 6], 3)
    12
    '''

    if _____________________________:

        _____________________________

    elif _____________________________:

        _____________________________

    with_elem = _____________________________ + __________

    without_elem = _____________________________________

    return _____________________________________________
```

```
def max_subset_sum(lst, n):
    if n == 0:
        return 0
    elif len(lst) <= n:
        return sum(lst)
    with_elem = max_subset_sum(lst[1:], n - 1) + lst[0]
    without_elem = max_subset_sum(lst[1:], n)
    return max(with_elem, without_elem)
```

Teaching Tips

- The solution to this problem is not the smartest nor the most elegant. It's a brute force approach using tree recursion.
- Tell students not to overthink the problem and approach it with tree recursion in mind.
- Drawing a tree diagram will be very helpful for understanding recursive calls.
- What are the base cases?
  - The simplest possible case is if `n = 0`, and there is nothing to sum.
  - Another case (read the spec carefully!) is if `n >= len(lst)`, and we return the sum of the whole list.
- What are the recursive cases?
  - We either choose to use an element in the subset or do not.
  - If we use the element, then add it to the sum and subtract `n`.
  - Either way, we have to shrink the list by the element we're considering, since elements can only be used at most once.
- How do we combine the recursive cases?
  - Make sure students understand that the recursive cases are numbers by the recursive leap of faith- `with_elem` is a number, not a function.
  - It's in the name- we use **`max`** to find the largest `subset_sum`.

Suggested Time: 10 min (if time!)

- This problem is more difficult, and optional for if you have time!